As you might imagine, you can build up considerably more advanced logic from there. A full description is at https://sinatrarb.com/intro.html, we'll cover some of the most important parts here.

We can start with the return value of the block. In our example it's a plain string. For more complex output, you'll likely want to use ERB or some other templating language. In Sinatra, you can do that with the erb method:

web/sinatra_01/sinatra_with_erb.rb

```
get "/" do
  erb(:root)
end
```

The first argument to erb is a symbol, and matches a filename, which is in a views subdirectory by default. So the previous snippet will render an ERB template in views/root.erb. Less commonly, the first argument can be a string, in which case the string is expected to be an ERB template itself, and is rendered as-is.

After the name of the template, you can pass in key/value pairs to erb as options. By default, the ERB template has access to instance variables set in the route, but if you want local variables to be visible you can set them with a locals: option. You can also set a layout with a layout: option—the layout is an ERB template that includes a call to <%= yield %> somewhere, at which point the actual template for the route is inserted. There are a few other options for erb in the official docs, and a very wide range of other template tools that are supported with their own similar helper methods.

The erb method also returns a string, you can also return an object that responds to each and returns a series of strings, in which case you are streaming output to the browser. You can also return a complete Rack response, meaning the entire three-element array, or you can return an integer, which is assumed to be an HTTP status code over an otherwise-empty response.

The route selectors can be made more flexible and complicated. You can match delete, get, link, options, patch, post, put, and unlink. You cannot by default have multiple HTTP verbs connected to the same route, though there are a couple of work-arounds for this including an extension called sinatra/multi_route.

In the route name, you can have parameters or wild cards. Parameters are either available in a hash called params or are passed as arguments to the block associated with the route.

web/sinatra_01/sinatra_with_param.rb

```
get "/user/:username" do |block_un|
  [
    "<p>Hello, #{params["username"]}</p>",
    "<p>#{block_un} has the same value</p>"
  ]
end
```

In this example we are taking advantage of our ability to return an object that responds to each—an array—and returns a series of strings. We like this slightly better than concatnating the strings together in the block.

The parameters are passed to the block in the order they are in the URL, and the block parameters can be named arbitrarily, but the params hash will pick up the name used in the path description. If the parameter name ends in a question mark, like :username? then the